# Nasir Book Agency POS System Documentation

This document provides complete technical and operational documentation for the Nasir Book Agency POS & Inventory Management System.

## 1. Project Overview

The Nasir Book Agency POS System is a full-stack bookstore management solution built to manage sales, invoices, customers, supplier records, stock, ledgers, payments, reports, and invoice generation.

## 2. Technology Stack

- Frontend: React + Vite
- Backend: Node.js + Express.js
- Database: MySQL
- Authentication: JWT
- Server: Ubuntu VPS
- Reverse Proxy: Nginx
- Process Manager: PM2
- SSL: Certbot + Let's Encrypt

## 3. Main Features

- Customer management
- Supplier management
- Book inventory management
- POS sales system
- Daily invoice merging for customers
- Walk-in customer handling
- Customer ledger tracking
- Payment recording
- Invoice editing
- Stock management
- JWT authentication
- Export functionality
- Pakistan timezone handling
- Production deployment with HTTPS

## 4. System Architecture

Frontend communicates with backend APIs using HTTPS.
Backend communicates with MySQL database.
Nginx serves frontend static files and proxies API requests to backend.

## 5. Folder Structure

/frontend → React frontend
/controllers → Backend controllers
/routes → API routes
/config → Database configuration
/middleware → JWT middleware
/models → Database models
/migrations → SQL migrations
/designs → Assets/design resources

## 6. Database Overview

Main tables:
- books
- customers
- suppliers
- sales
- sale_items
- payments

## 7. Authentication Flow

Users authenticate using JWT tokens.
Frontend stores token in localStorage.
Protected routes require Authorization Bearer token.

## 8. Sales Workflow

1. User selects customer
2. User adds books
3. Stock validation occurs
4. Invoice created or merged for same-day customer sales
5. Ledger updated
6. Invoice generated

## 9. Daily Invoice Merge Logic

For registered customers:
- Same customer + same day = same invoice
- Same book again = quantity updated
- Different books = appended to invoice

For walk-in customers:
- Always separate invoices

## 10. Stock Management

Stock decreases on sales.
Stock restores on invoice edits/removals.
FOR UPDATE queries are used to reduce concurrency issues.

## 11. Invoice System

Invoices include:
- Invoice number
- Customer information
- Previous remaining balance
- Current invoice total
- Updated balance
- Itemized books

## 12. Customer Ledger

Tracks:
- Sales
- Payments
- Running balances
- Invoice history

## 13. Deployment Architecture

Production server uses:
- Ubuntu VPS
- PM2 for backend process management
- Nginx reverse proxy
- SSL using Let's Encrypt

## 14. Production Environment Variables

Backend .env:
PORT=5002
DB_HOST=127.0.0.1
DB_USER=nasiruser
DB_PASSWORD=********
DB_NAME=nasirbook
JWT_SECRET=********

## 15. Frontend Environment

Frontend .env:
VITE_API_URL=https://nasirbookagency.com

## 16. VPS Deployment Steps

1. Clone repository
2. Install dependencies

3. Configure .env
4. Import MySQL database
5. Build frontend
6. Configure Nginx
7. Configure SSL
8. Start PM2 services

## 17. GitHub Update Workflow

Local:
git add .
git commit -m 'message'
git push

VPS:
git pull
npm run build
pm2 restart nasirbook-api

## 18. Security Measures

- JWT authentication
- HTTPS SSL encryption
- Protected routes
- Environment variables
- Nginx reverse proxy

## 19. Scalability Considerations

- Indexed database columns
- PM2 process management
- Reverse proxy architecture
- Modular controller structure
- Optimized invoice merge logic

## 20. Future Improvements

- Multi-user roles
- Barcode scanning
- Automated backups
- Analytics dashboard
- Docker deployment
- CI/CD automation

## 21. Troubleshooting

- PM2 logs: pm2 logs nasirbook-api
- Restart backend: pm2 restart nasirbook-api

- Restart nginx: systemctl restart nginx
- Build frontend: npm run build

## 22. Conclusion

The Nasir Book Agency POS System is a production-ready bookstore ERP/POS platform capable of handling inventory, invoicing, ledger tracking, and sales operations with scalable deployment architecture.